KIEU STUDIO

12 mẫu tư duy giải thuật toán

Cheatsheet tiếng Việt: nhận ra bài thuộc dạng nào, nhớ khung code Python 3, và luyện cùng AI mà không học vẹt.

Hash Map · Two Pointers · Sliding Window · Prefix Sum · Binary Search · Monotonic Stack · BFS · Backtracking · Heap / Top-K · Quy hoạch động · Khoảng & Tham lam · Union-Find

Dùng tài liệu này thế nào?

1

Đọc đề, **chưa code**. Dò mục "Dấu hiệu" để đoán bài thuộc mẫu nào.

2

Tự viết lại khung code từ trí nhớ, rồi mới so với khung trong tài liệu.

3

Làm bài luyện theo thứ tự dễ → khó. Kẹt quá 25 phút thì dùng prompt ở trang 6.

Biên soạn: Anthony Kieu · Kieu Studio · kieu.app
Miễn phí, được chia sẻ lại nếu giữ nguyên nguồn.

01 Hash Map — đổi bộ nhớ lấy thời gian

DẤU HIỆU

"Có cặp/phần tử nào thoả…", cần tra cứu nhanh "đã gặp chưa", đếm tần suất, gom nhóm.

Ý TƯỞNG

Thay vòng lặp thứ hai bằng một lần tra cứu O(1) trong dict/set.

ĐỘ PHỨC TẠP

Thời gian O(n), bộ nhớ O(n).

```
def two_sum(nums, target):
    pos = {}                    # giá trị -> chỉ số
    for i, x in enumerate(nums):
        if target - x in pos:
            return [pos[target - x], i]
        pos[x] = i
```

Luyện: LeetCode 1 Two Sum · 49 Group Anagrams · 128 Longest Consecutive Sequence

02 Two Pointers — hai đầu tiến lại gần nhau

DẤU HIỆU

Mảng đã sắp xếp (hoặc sắp xếp được), tìm cặp/bộ ba, so sánh hai đầu, đảo chuỗi.

Ý TƯỞNG

Mỗi bước loại bỏ được một đầu chắc chắn không cho kết quả tốt hơn.

ĐỘ PHỨC TẠP

Thời gian O(n), bộ nhớ O(1).

```
def max_area(h):
    l, r, best = 0, len(h) - 1, 0
    while l < r:
        best = max(best, min(h[l], h[r]) * (r - l))
        if h[l] < h[r]:
            l += 1              # cột thấp giới hạn diện tích
        else:
            r -= 1
    return best
```

Luyện: 167 Two Sum II · 11 Container With Most Water · 15 3Sum · 42 Trapping Rain Water

03 Sliding Window — cửa sổ co giãn

DẤU HIỆU

"Chuỗi con / mảng con liên tiếp dài nhất / ngắn nhất thoả điều kiện…"

Ý TƯỞNG

Mở rộng bên phải; khi cửa sổ vi phạm điều kiện thì thu hẹp bên trái.

ĐỘ PHỨC TẠP

Thời gian O(n), bộ nhớ O(số ký tự khác nhau).

```
def longest_unique(s):
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if last.get(ch, -1) >= left:
            left = last[ch] + 1  # nhảy qua ký tự trùng
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

Luyện: 3 Longest Substring Without Repeating Characters · 209 Minimum Size Subarray Sum · 76 Minimum Window Substring

04 Prefix Sum — tổng dồn để trả lời nhanh

DẤU HIỆU

"Tổng của đoạn [i, j]", "có bao nhiêu mảng con có tổng bằng k" (kể cả số âm).

Ý TƯỞNG

sum(i..j) = P[j] – P[i–1]. Kết hợp Hash Map để đếm các P đã gặp.

ĐỘ PHỨC TẠP

Thời gian O(n), bộ nhớ O(n).

```
def subarray_sum(nums, k):
    count, s, ans = {0: 1}, 0, 0
    for x in nums:
        s += x
        ans += count.get(s - k, 0)
        count[s] = count.get(s, 0) + 1
    return ans
```

Luyện: 303 Range Sum Query – Immutable · 560 Subarray Sum Equals K · 238 Product of Array Except Self

05 Binary Search — tìm vị trí "đúng" đầu tiên

DẤU HIỆU

Dữ liệu sắp xếp, hoặc câu hỏi kiểu "giá trị nhỏ nhất sao cho … vẫn làm được" (tìm kiếm trên đáp án).

Ý TƯỞNG

Viết hàm ok(x) có dạng Sai…Sai Đúng…Đúng, rồi tìm chữ Đúng đầu tiên.

ĐỘ PHỨC TẠP

O(log M) lần gọi ok(x).

```
def first_true(lo, hi, ok):
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid             # mid có thể là đáp án
        else:
            lo = mid + 1
    return lo

# Koko: tốc độ ăn nhỏ nhất kịp trong h giờ
# first_true(1, max(piles),
#   lambda k: sum((p + k - 1) // k for p in piles) <= h)
```

Luyện: 704 Binary Search · 33 Search in Rotated Sorted Array · 875 Koko Eating Bananas · 4 Median of Two Sorted Arrays

06 Monotonic Stack — "phần tử lớn hơn tiếp theo"

DẤU HIỆU

"Bao lâu nữa thì gặp giá trị lớn/nhỏ hơn", ngoặc hợp lệ, hình chữ nhật lớn nhất.

Ý TƯỞNG

Ngăn xếp giữ các chỉ số "đang chờ câu trả lời". Phần tử mới giải quyết những phần tử nhỏ hơn nó.

ĐỘ PHỨC TẠP

O(n): mỗi phần tử vào và ra ngăn xếp đúng một lần.

```
def daily_temps(t):
    ans, st = [0] * len(t), []
    for i, x in enumerate(t):
        while st and t[st[-1]] < x:
            j = st.pop()
            ans[j] = i - j
        st.append(i)
    return ans
```

Luyện: 20 Valid Parentheses · 739 Daily Temperatures · 84 Largest Rectangle in Histogram

07 BFS — đường ngắn nhất theo số bước

DẤU HIỆU

"Ít bước nhất", lan theo từng lớp (cây theo tầng, lưới, cam thối lan).

Ý TƯỞNG

Hàng đợi xử lý hết lớp hiện tại rồi mới sang lớp tiếp theo; đánh dấu đã thăm khi đưa vào hàng đợi.

ĐỘ PHỨC TẠP

O(V + E).

```
from collections import deque

def bfs(start, goal, nbrs):
    q, seen, steps = deque([start]), {start}, 0
    while q:
        for _ in range(len(q)):   # một lớp
            u = q.popleft()
            if u == goal:
                return steps
            for v in nbrs(u):
                if v not in seen:
                    seen.add(v)
                    q.append(v)
        steps += 1
    return -1
```

Luyện: 102 Binary Tree Level Order Traversal · 200 Number of Islands · 994 Rotting Oranges · 127 Word Ladder

08 DFS / Backtracking — thử, đi sâu, quay lui

DẤU HIỆU

"Liệt kê tất cả" tập con, hoán vị, tổ hợp; tìm đường trong lưới.

Ý TƯỞNG

Chọn → đệ quy → bỏ chọn. Cắt nhánh sớm khi chắc chắn không ra kết quả.

ĐỘ PHỨC TẠP

Tập con: O(n · 2^n).

```
def subsets(nums):
    res = []
    def bt(start, path):
        res.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])     # chọn
            bt(i + 1, path)          # đi sâu
            path.pop()               # quay lui
    bt(0, [])
    return res
```

Luyện: 78 Subsets · 46 Permutations · 39 Combination Sum · 79 Word Search

09 Heap / Top-K — giữ lại K phần tử tốt nhất

DẤU HIỆU

"K lớn nhất / nhỏ nhất / thường gặp nhất", trộn nhiều danh sách đã sắp xếp.

Ý TƯỞNG

Min-heap kích thước k: phần tử nhỏ nhất trong heap chính là phần tử lớn thứ k.

ĐỘ PHỨC TẠP

O(n log k), bộ nhớ O(k).

```
import heapq

def kth_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)     # bỏ phần tử nhỏ nhất
    return h[0]
```

Luyện: 215 Kth Largest Element in an Array · 347 Top K Frequent Elements · 23 Merge k Sorted Lists

10 Quy hoạch động — bài lớn từ bài nhỏ

DẤU HIỆU

"Số cách", "nhỏ nhất / lớn nhất", lựa chọn ở bước này ảnh hưởng bước sau, bài con lặp lại.

Ý TƯỞNG

Định nghĩa dp[trạng thái], viết công thức chuyển, chọn thứ tự tính sao cho bài con có trước.

ĐỘ PHỨC TẠP

Số trạng thái × chi phí mỗi lần chuyển. Coin Change: O(amount · số loại xu).

```
from math import inf

def coin_change(coins, amount):
    dp = [0] + [inf] * amount    # dp[a]: số xu ít nhất
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] < inf else -1
```

Luyện: 70 Climbing Stairs · 198 House Robber · 322 Coin Change · 1143 Longest Common Subsequence · 10 Regular Expression Matching · 44 Wildcard Matching

11 Khoảng & Tham lam — sắp xếp rồi quét

DẤU HIỆU

Danh sách [bắt đầu, kết thúc]: gộp, đếm chồng lấn, xếp lịch; "bỏ ít nhất bao nhiêu".

Ý TƯỞNG

Sắp xếp theo điểm đầu (hoặc điểm cuối), rồi quyết định tại chỗ với khoảng gần nhất.

ĐỘ PHỨC TẠP

O(n log n) cho bước sắp xếp.

```
def merge(iv):
    iv.sort()
    out = [iv[0][:]]
    for s, e in iv[1:]:
        if s <= out[-1][1]:          # chồng lấn
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

Luyện: 56 Merge Intervals · 435 Non-overlapping Intervals · 55 Jump Game

12 Union-Find — gom nhóm các phần tử liên thông

DẤU HIỆU

"Có bao nhiêu nhóm", "thêm cạnh này có tạo chu trình không".

Ý TƯỞNG

Mỗi nhóm có một đại diện; gộp hai nhóm bằng cách nối đại diện.

ĐỘ PHỨC TẠP

Nén đường đi: O(log n) mỗi thao tác (khấu hao); thêm gộp theo hạng thì gần như O(1).

```
parent = list(range(n))

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]   # nén đường đi
        x = parent[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False             # đã cùng nhóm → chu trình
    parent[ra] = rb
    return True
```

Luyện: 547 Number of Provinces · 684 Redundant Connection · 207 Course Schedule (tô-pô)

Luyện cùng AI mà không học vẹt

AI giải hộ bạn trong 3 giây, nhưng như vậy bạn không học được gì. Hãy dùng AI như một người hướng dẫn: gợi ý từng chút, bắt bạn tự giải thích.

1. **Tự làm 20–25 phút trước.** Viết ra: bài thuộc mẫu nào, vì sao, và một ví dụ nhỏ tự tạo.
2. **Xin gợi ý, không xin lời giải.** Mỗi lần chỉ hỏi một gợi ý.
3. **Tự viết code**, rồi nhờ AI tìm trường hợp biên làm code sai.
4. **Giải thích lại cho AI** như đang dạy người khác. AI chấm xem bạn hiểu đúng chưa.
5. **Làm lại sau 3 ngày**, không xem bài cũ.

> **Prompt 1 — Gợi ý từng bước**
> Mình đang giải LeetCode {số}. Đừng đưa lời giải hay code. Hãy cho mình đúng MỘT gợi ý nhỏ nhất để đi tiếp. Mình sẽ hỏi tiếp nếu vẫn kẹt.

> **Prompt 2 — Nhận dạng mẫu**
> Đây là đề bài: {dán đề}. Mình nghĩ nó thuộc mẫu {tên mẫu} vì {lý do}. Lập luận của mình đúng hay sai ở đâu? Chưa cần nói mẫu đúng là gì.

> **Prompt 3 — Tìm trường hợp biên**
> Đây là code của mình: {code}. Đừng sửa code. Hãy liệt kê 5 input có khả năng làm code sai (mảng rỗng, phần tử trùng, số âm, giá trị lớn…) và để mình tự chạy thử.

> **Prompt 4 — Kiểm tra mức hiểu**
> Mình sẽ giải thích lời giải bằng lời của mình: {giải thích}. Hãy hỏi mình 3 câu để kiểm tra mình hiểu thật hay chỉ nhớ. Sau đó chấm điểm 1–10 và chỉ ra chỗ còn mơ hồ.

Lộ trình 4 tuần gợi ý

TUẦN	MẪU	MỤC TIÊU
1	01 Hash Map · 02 Two Pointers · 03 Sliding Window	Nhận dạng được mẫu trong 2 phút đầu
2	04 Prefix Sum · 05 Binary Search · 06 Monotonic Stack	Tự viết template không cần nhìn
3	07 BFS · 08 Backtracking · 09 Heap	Giải bài Medium trong 30 phút
4	10 Quy hoạch động · 11 Khoảng · 12 Union-Find	Giải thích lời giải trong 3 phút như khi phỏng vấn

Học tiếp cùng Kieu Studio & Kieu Academy

Mỗi mẫu trong tài liệu này đều có (hoặc sắp có) một video giải thích từng bước bằng tiếng Việt trên kênh Kieu Studio: đề bài, ví dụ, gợi ý, 10 giây để bạn tự nghĩ, rồi mới tới lời giải.

Muốn học có lộ trình và có người kèm?

Kieu Academy dạy lập trình và AI theo hướng thực hành: từ tư duy giải bài tới dùng AI để xây công cụ thật cho công việc.

KHOÁ HỌC & WORKSHOP
kieu.app/teaching

VIDEO MIỄN PHÍ MỖI TUẦN
youtube.com/@kieugroup

Số bài LeetCode được trích để luyện tập; đề bài thuộc về LeetCode. Code mẫu trong tài liệu do Kieu Studio viết và đã chạy thử với các ví dụ trong đề.